Information and Software Technology 185 (2025) 107786

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Generating vulnerability security fixes with Code Language Models

Guru Bhandari *, Nikola Gavric, Andrii Shalaginov

School of Economics, Innovation, and Technology, Kristiania University of Applied Sciences, Kirkegata 24, Oslo, 0153, Norway

ARTICLE INFO

Dataset link: https://github.com/SmartSecLab/PatchLM, https://zenodo.org/records/10955342, https://huggingface.co/datasets/ASSERT-KTH/repairllama-datasets

Keywords:
Large language models
Automated program repair
Code Language Models
Patch generation
Security vulnerability
Source code

ABSTRACT

Existing Code Language Models (CLM) have demonstrated significant potential in several coding tasks, including automated code generation in software engineering. Similarly, Automated Program Repair (APR) has shown considerable progress in addressing general software vulnerabilities, yet its application utilizing the recently developed CLM remains unexplored. In this paper, we introduce Patch Language Model (PatchLM), a novel CLM fine-tuned to fix security vulnerabilities in code blocks retrieved from commit hunks associated with Common Vulnerabilities and Exposures (CVE) records. Our proposed model leverages CLM to understand secure coding practices and generate accurate patches. The study aims to address the diverse nature of security flaws across multiple programming languages. Our experimental evaluation demonstrated that PatchLM significantly outperforms the baseline *CodeT5* and *CodeLlama* models in generating effective security patches, as reflected in the performance metrics. Specifically, PatchLM achieves improvements of up to 48.35% in CodeBLEU and 28.9% in ROUGE scores compared to the baseline models. Our study demonstrates the practicality and significance of PatchLM in generating vulnerability repairs, providing valuable support for under-resourced security analysts, and paving the way for future research in automated vulnerability fixing with CLM.

1. Introduction

In 2024, the National Vulnerability Database (NVD) recorded its highest annual figure, with over 38,783 new common IT security vulnerabilities and exposures (CVE) reported so far [1]. However, the NVD now faces significant challenges, including a 75% backlog of unprocessed vulnerabilities in 2024, driven by increased submissions, outdated infrastructure, and budget constraints [2]. These delays hinder timely vulnerability remediation and impact overall security. To address these issues, industry efforts focus on modernization initiatives and leveraging Artificial Intelligence (AI) to prioritize risks and streamline vulnerability management, enhancing security defenses.

Exploring security vulnerability fixes presents a promising research direction for Automated Program Repair (APR) techniques [3]. While APR tools have seen significant development in addressing general bugs in recent years, their application to security flaws or vulnerabilities remains relatively under-researched. Addressing security vulnerabilities could provide a promising research path for APR techniques [4]. The complexity of security-related bugs, the lack of comprehensive datasets, and the diverse nature of security flaws are among the significant challenges to addressing security vulnerabilities in APR techniques.

The robust capabilities of the Large Language Model (LLM) in coding, such as Copilot [5], AlphaCode [6], *CodeLlama* [7], have led to their widespread use among developers for recommending code snippets, significantly improving productivity and experience, especially for senior developers [8]. Recently, tools leveraging LLMs [9–11] for program repair and testing have produced highly accurate and satisfactory results, suggesting a deep understanding of the semantics of the program.

According to the survey in [12], code-focused LLM or Code Language Model (CLM) can help address security challenges. CLM can be applied to generate potential fixes for identified vulnerabilities by leveraging the capabilities of such models to understand secure coding practices and existing code patterns, thereby aiding in the automation of repair tasks. Analyzing source code allows for the early detection of security vulnerabilities, enabling developers to address problems before they can be exploited in a production environment [13]. The CLM on the source code thoroughly examines the entire codebase, ensuring that even subtle and hidden vulnerabilities are identified and mitigated. Identifying and fixing vulnerabilities at the source code level is often more cost-effective than dealing with security breaches or patching vulnerabilities in deployed software [14].

Extensive research in APR has focused on programming language-specific fixes, including languages such as C/C++ [10], Java [3,15,16], PHP [4], and JavaScript [17]. However, there is a notable research gap in identifying and predicting source code patches in multiple programming languages [18]. This gap is primarily due to the absence

* Corresponding author.
E-mail address: guru.bhandari@kristiania.no (G. Bhandari).

https://doi.org/10.1016/j.infsof.2025.107786
Received 4 September 2024; Received in revised form 21 April 2025; Accepted 12 May 2025
Available online 28 May 2025
0950-5849/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Table 1
Abbreviations.

Abbreviation	Description
AI	Artificial Intelligence
APR	Automated Program Repair
AST	Abstract Syntax Tree
BERT	Bidirectional Encoder Representations from Transformers
BLEU	BiLingual Evaluation Understudy
BPE	Byte Pair Encoding
CLM	Code Language Model
CodeGen	Code Generation
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness and Enumeration
DL	Deep Learning
GPT	Generative Pretrained Transformer
LCS	Longest Common Subsequence
LLM	Large Language Model
NLP	Natural Language Processing
NMT	Neural Machine Translation
ROUGE	Recall-Oriented Understudy for Gisting Evaluation

of comprehensive benchmark datasets and domain-specific CLM covering a variety of languages. Specifically, vulnerability fixes require the generation of patches to improve the coverage of common security vulnerabilities. For that, a CLM can be utilized to understand the context of a vulnerability and generate the correct patches to fix it. Moreover, recent studies have explored Neural Machine Translation (NMT)-based approaches for automated vulnerability repair. Despite progress, these methods are hindered by significant limitations, such as challenges in accurately predicting repairs and addressing diverse vulnerability types [19].

In this work, we propose PatchLM - a security vulnerability fixing CLM fine-tuned on code block of commit hunks based on CVE records. The in-context fine-tuned model helps generate automated security patches for vulnerable samples of multiple programming languages. The measured performance scores show that our proposed model outperforms the baseline *CodeT5* and *CodeLlama* models for generating security patches of their corresponding vulnerabilities.

The major contributions of the study can be summarized as follows:

- Proposal of PatchLM as an introduction of a new CLM fine-tuned for fixing security vulnerabilities.
- Automated security patch generation demonstrates the model's ability to generate security patches for the identified vulnerabilities.
- Empirical evidence shows that PatchLM outperforms the baseline models in generating effective security patches.

The remainder of the paper is structured as follows: Section 2 provides an overview of existing research in deep learning and CLM-based APR techniques, mainly focusing on their application to security vulnerability fixing. It also discusses the relevant work on CLMs and their role in automated vulnerability fixing. Section 3 describes the methodology used for developing the proposed PatchLM model, which includes details on data collection, preprocessing steps, prompt engineering, and fine-tuning process. The section also summarizes the evaluation metrics for comparing baseline CLMs and the proposed model. Section 4 presents the results and performance measures to evaluate our instruct models. Similarly, Section 5 provides a detailed discussion of the study, highlighting the strengths and limitations of the proposed model, including the future research directions in the field of APR for security vulnerabilities. Finally, Section 6 summarizes the key findings of the research, reiterating the paper's contributions. Table 1 presents the abbreviations used in the paper.

2. Background and related work

In this section, we introduce key concepts and provide an overview of existing research in CLM, mainly focusing on their application to fixing security vulnerabilities. The section also discusses the relevant work on deep learning (DL) and natural language processing (NLP) models and their role in the automated fixing of security vulnerabilities.

Like in other text translation problems, language models can be used for software security and patch generation. Language models, especially those based on transformer architectures like Generative Pretrained Transformer (GPT), have shown great promise in software development [20]. The large learning models can be trained on extensive codebases to grasp the syntax and semantics of programming languages. With this understanding, they can generate new code, suggest completions, identify bugs, and even create patches. Table 2 summarizes the major LLMs/CLMs trained for code understanding with their capabilities and language support.

2.1. CLM for software security

Recently, code-focused large language models (CLMs) have demonstrated significant potential in code completion and code generation from natural language descriptions [27]. Below, we provide an overview of the CLMs that are well-suited for code understanding and prompting tasks, which we have employed for security patch generation tasks in this study.

2.1.1. CodeT5 family

The *CodeT5* [21] is a new family of open-source LLM designed for code understanding and generation tasks. *CodeT5* is a transformer-based model specifically pre-trained for tasks involving multiple programming languages. The model was built upon the *T5* (Text-to-Text Transfer Transformer) architecture [31], adapting it for code-related tasks. The *CodeT5* model was trained on large-scale code datasets sourced from GitHub repositories that include a mix of natural language (e.g., comments, documentation) and programming code. The model features an encoder–decoder architecture that supports various modes of operation, including encoder-only, decoder-only, and encoder–decoder. To train CodeT5+ [22], a diverse set of pretraining tasks is employed, such as span denoising, causal language modeling, contrastive learning, and text-code matching. These tasks enable the model to learn rich representations from both unimodal code data and bimodal code-text data. This comprehensive approach results in a versatile model that can effectively understand and generate code from human-readable descriptions.

2.1.2. CodeLlama model

CodeLlama [23] is a family of LLMs specially designed for coding tasks developed on the Llama 2 framework. The model can generate code and natural language explanations related to code. The *CodeLlama* supports many popular programming languages, including Python, C++, Java, PHP, TypeScript (JavaScript), C#, Bash, and others. *CodeLlama* is available in several specialized versions [32] to cater to different needs: the foundational *CodeLlama* models, Python-specialized *CodeLlama-Python*, and instruction-following *CodeLlama-Instruct* models. Each variant is available in four sizes based on the number of parameters (the building blocks of an AI model that determine its complexity and capabilities). The model with 7 billion (B) parameters is suitable for smaller and faster, less resource-intensive tasks. The 13B, 34B, and 70B parameters are for larger versions with higher accuracy and better performance, though they require more computational resources.

2.2. Existing studies on CLM-based APR

With the advancements of artificial intelligence, APR techniques adapt the DL and NLP models [3,28,30,33–36] to take a buggy code as input and generate a patched version of the code. APR technique helps developers improve software reliability and maintainability by generating automated patches to repair software defects [37]. The researchers

Table 2
CLMs for code-related tasks and their supported programming languages.

Model	Year	Tasks Supported (on code)	Programming Languages
CodeT5 [21]	2021	Understanding, generation, translation, summarization	Python, Java, JavaScript, PHP, Ruby, Go
CodeT5+ [22]	2023	Understanding, generation, completion, bug detection	Python, Java, JavaScript, PHP, Ruby, Go
CodeLlama [23]	2024	Generation, completion, summarization	Python, C++, Java, Bash, C#, PHP, TypeScript, HTML
Codex [13]	2021	Generation, completion, natural language to code, translation	Python, JavaScript, Go, Perl, PHP, Ruby, Swift, TypeScript, BASH
CodeBERT [24]	2020	Search, summarization, documentation generation	Python, Java, JavaScript, PHP, Ruby, Go
GraphCodeBERT [25]	2021	Search, summarization, translation	Python, Java, JavaScript, PHP, Ruby, Go
AlphaCode [26]	2022	Generation for competitive programming	Python, C++
PolyCoder [27]	2022	Generation, completion	Python, JavaScript, C, C++, Java, PHP, Ruby, Go, TypeScript
PLBART [28]	2021	Summarization, generation, translation	Python, Java, JavaScript, PHP, Ruby, Go
CuBERT [29]	2020	Understanding, bug detection, search	Python
CodeGen [30]	2023	Generation, completion	Python, JavaScript, Java, PHP, Ruby, Go, TypeScript

have introduced learning-based APR tools that treat program repair as a Neural Machine Translation (NMT) problem. With the advent of LLM, there has been a focus on integrating LLM into APR tools to enhance their patching capabilities and reduce costs associated with time and resources. Moreover, the existing studies have highlighted that security patches often require unique fix patterns, such as method calls with input validation. This suggests the need for enhanced repair patterns in APR tools to address security vulnerabilities better [3].

Fu et al. [19] proposed VulRepair, a T5-based automated software vulnerability repair method leveraging the pre-training Byte Pair Encoding (BPE) components, which covers 745 out of 1706 real-world well-known vulnerabilities (e.g., Use After Free, Improper Input Validation, OS Command Injection). Similarly, VRepair [38] employs a standard Encoder-Decoder Transformer model [39] for performing vulnerability repairs. Initially, VRepair is trained on a labeled bug-fixing dataset and subsequently fine-tuned on a dedicated vulnerability dataset to generate accurate vulnerability fixes. Additionally, Zhou et al. [40] proposed a transformer-based neural network designed for vulnerability repair, namely VulMaster, on a C/C++ vulnerability repair dataset containing 1754 projects and 5800 vulnerable functions. Their study has considered code similarity measurement metrics, including exact match, BLEU (Bilingual Evaluation Understudy), and CodeBLEU.

A study by De-Fitero-Dominguez et al. [41] also explored automated code vulnerability repair techniques using two *CodeLlama* and *Mistral* [42] models fine-tuned on C/C++ vulnerability data. The study claims it outperforms VulRepair [19], VRepair [38], and VulMaster [40]. However, the study does not provide the open-source code and the fine-tuned models. All of these studies in the paragraph utilize C/C++ code parts of two vulnerability datasets: CVEfixes [43] and BigVul [44], which are primarily for vulnerability detection rather than patch generation tasks.

The APR4Vul paper [3] evaluates nine state-of-the-art APR tools and one vulnerability-specific tool to assess their effectiveness in generating patches for 79 real-world Java vulnerabilities from the Vul4J dataset. The study finds that while APR tools can generate testable patches for about 20% vulnerabilities, only 44% of these patches successfully fix security bugs without compromising functionality. Similarly, Hendrycks et al. [45] introduced the APPS dataset and assessed the code generation capabilities of various Generative Pre-trained Transformer (GPT) model variants fine-tuned on this dataset. Following this, several language model-based platforms for automatic code generation have emerged, including Codex [13], the back-end model that powers GitHub Copilot [5], as well as Alphacode [21,26], CodeBERT [24], GraphCodeBERT [25], and [46]. Table 2 also summarizes the code-specific CLMs for code-related tasks and their supported programming languages.

Most of the existing work emphasizes deep learning techniques with fresh training on small data that would not represent the common vulnerability patches. Moreover, the approaches lack the standard semantics and syntactic information of general code. Most of the existing studies only utilize data from specific programming languages and specific granularity levels of programs, mostly function- or file-level. Unlike the existing studies, we applied pre-trained models that are suitable for coding tasks and fine-tuning on commit patch hunks, which is the lightweight information of the vulnerabilities. The fine-tuned models outperform the pre-trained base models and can be utilized for better and more secure coding practices. The fine-tuning was performed on language-specific and combined vulnerability-fix pairs data.

3. Methodology

This section describes the methodology for developing the proposed PatchLM model, which includes the LLM pipeline covering data collection, preprocessing steps, and fine-tuning the CLM on security commit hunks. In this work, the term *instruct model* refers specifically to the models we have fine-tuned based on the base models. While models like CodeLLaMA-7B-Instruct are already instruction-tuned, our use of *instruct model* denotes our customized fine-tuned versions, rather than the pre-existing instruction-tuned models. The section also presents the experimental setup used to assess the performance of the fine-tuned models.

The proposed framework, as shown in Fig. 1, outlines an innovative approach to automated patch generation within software security. This method involves fine-tuning CLMs using source code snippets. The used dataset comprises pairs of code segments, one representing the vulnerability and the other its corresponding fix, recorded in the

CVE vulnerability database. Moreover, the source code snippets are gathered from the repositories, and their specific commits are from the CVE reference links. The aggregated dataset is the foundation for fine-tuning CLMs, a process crucial for automatically generating patches. Initially, established code-focused models such as *CodeT5*, and *CodeLlama* were employed. These models were then fine-tuned using the collected dataset and evaluated against a designated evaluation set. Evaluation metrics such as the BLEU (BiLingual Evaluation Understudy) and ROUGE (Recall Oriented Understudy for Gisting Evaluation) measures are employed to assess the efficacy of the fine-tuned models. These measures serve as a benchmark, enabling a comparative analysis of the performance of the fine-tuned models in patch generation against their original counterparts. The patch generation task can be formally defined as follows.

Task Definition: Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ represent a dataset of n code snippet pairs, where x_i is a vulnerable code snippet, and y_i is its corresponding fixed version. The objective is to train a model $\mathcal{M}$, such that for a given vulnerable code snippet x, the model predicts a fix $\hat{y}$ that closely approximates y, minimizing the loss function, $\mathcal{L}(y, \hat{y})$ according to defined metrics, as defined in Eq. (1):

$$\mathcal{L}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_{\text{CE}}(y_i, \mathcal{M}(x_i)), \tag{1}$$

where, $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss [47] between the predicted and ground truth outputs.

In summary, a CLM pipeline is a structured workflow used to train, fine-tune, deploy, and utilize the model. The pipeline typically involves multiple stages, each focusing on a specific aspect of the model's lifecycle, from data preprocessing to model inference. Algorithm 1 presents the summary of the proposed CLM pipeline. Moreover, the following points outline the major steps of the proposed CLM.

1. *Data collection and preprocessing:* We load the dataset D from the given sources as mentioned in the dataset description subsection (ref. Section 3.4). The dataset has vulnerable-fix pairs to fine-tune the models. The preprocessing stage involves — loading the dataset and preparing the examples to feed into the model, adding missing information, for example, questions, programming languages, and splitting the dataset into different parts for fine-tuning and evaluating the resulting models.
2. *Model training:* In this study, we didn't perform fresh training of the CLM. Instead, we load the base pre-trained models from the *Salesforce/codet5-base* [48] for the *CodeT5* model and pre-existing instruction-tuned *codellama/CodeLlama-7b-Instruct-hf* [49] for the *CodeLlama* model for fine-tuning them for the patch generation task. Similarly, the corresponding tokenizer is loaded for the fine-tuning and evaluation of the models. In the case of *CodeLlama*, the model is quantized to 4 bit to make it more efficient for the resources ($\mathcal{M}_{\text{CodeLlama}}$, 4 bits). We load the original model for *CodeT5* since it's not as heavy as the *CodeLlama* model.
3. *Model fine-tuning:* After loading the pre-trained models, which are for the general purpose multi-tasks code understanding and prompting, we have performed the domain-specific fine-tuning on $\mathcal{D}_{\text{train}}$ set of the data. The bigger model may be fine-tuned on a smaller, domain-specific dataset to improve performance on specific task, i.e., automated patch generation in this study.
4. *Evaluation and validation:* The model's performance is assessed using LLM-specific measures like BLEU scores and ROUGE scores as mentioned in Section 4.1. The performance metrics measure the similarity between predicted (y) and reference ($\hat{y}$) code snippets. The model is validated on a set D_{val} of dataset D (different than the training set) to ensure that it generalizes well to new data and does not overfit.
5. *Deployment:* The fine-tuned instruct model produced in the study can be deployed to a production environment where it can handle real-time queries related to the source code vulnerability assessments, especially automated patch generation. The produced models are domain-specific and fine-tuned *CodeLlama* models are optimized with 4-bit quantization for efficiently handling high loads with low latency.

Algorithm 1 Fine-tuning and deployment of CLM for automated security patch generation

Input: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$, where x_i is the input source code, and y_i is the corresponding fix
Output: Fine-tuned models $\mathcal{M}_{\text{fine-tuned}}$ ($\mathcal{M}^{\text{fine-tuned}}_{\text{CodeLlama}}$ and $\mathcal{M}^{\text{fine-tuned}}_{\text{CodeT5}}$)

begin Data Collection and Preprocessing
  Load dataset $\mathcal{D}$
  Collect vulnerable-fix pairs from D as
  $\mathcal{D}_{\text{pairs}} = \{(x_i, y_i)\}_{i=1}^{m},\ m \leq n$
  begin preprocess dataset:
    Format examples for model input
    Add missing information, i.e, questions, programming languages
    Split $\mathcal{D}_{\text{pairs}}$ dataset into training set $\mathcal{D}_{\text{train}}$ and validation set $\mathcal{D}_{\text{val}}$

begin Model Loading and Preparation
  Load pre-trained models:
  $\mathcal{M}_{\text{CodeT5}} \leftarrow$ Load(Codet5)
  $\mathcal{M}_{\text{CodeLlama}} \leftarrow$ Load(CodeLlama)
  Load corresponding tokenizers: $\mathcal{T}_{\text{CodeT5}}$ and $\mathcal{T}_{\text{CodeLlama}}$
  Quantize model:
  $\mathcal{M}^{4b}_{\text{CodeLlama}} \leftarrow$ Quantize($\mathcal{M}_{\text{CodeLlama}}$, 4 bits)

begin Model Fine-Tuning
  Fine-tune the baseline models on $\mathcal{D}_{\text{train}}$:
  $\mathcal{M}^{\text{fine-tuned}}_{\text{CodeT5}} \leftarrow$ FineTune($\mathcal{M}_{\text{CodeT5}}, \mathcal{D}_{\text{train}}$)
  $\mathcal{M}^{\text{fine-tuned}}_{\text{CodeLlama}} \leftarrow$ FineTune($\mathcal{M}^{fine-tuned(4b)}_{\text{CodeLlama}}, \mathcal{D}_{\text{train}}$)

begin Evaluation and Validation
  begin Evaluate models
    $S_{\text{CodeT5}} = \text{Evaluate}(\mathcal{M}^{\text{fine-tuned}}_{\text{CodeT5}}, \mathcal{D}_{\text{val}})$
    $S_{\text{CodeLlama}} = \text{Evaluate}(\mathcal{M}^{\text{fine-tuned}}_{\text{CodeLlama}}, \mathcal{D}_{\text{val}})$
  Validate the models on validation set $\mathcal{D}_{\text{val}}$

begin Deployment [optional]
  Deploy fine-tuned models $\mathcal{M}^{\text{fine-tuned}}_{\text{CodeT5}}$ and $\mathcal{M}^{\text{fine-tuned}}_{\text{CodeLlama}}$
  Optimize deployment, $\mathcal{M}^{fine-tuned(4b)}_{\text{CodeLlama}}$
return $\mathcal{M}^{\text{fine-tuned}}_{\text{CodeLlama}}$ and $\mathcal{M}^{\text{fine-tuned}}_{\text{CodeT5}}$

3.1. Prompt engineering

We need to follow the structured prompt so that the CLM model can effectively learn to generate the output code. Therefore, the base models require structured prompt engineering to learn the context, fine-tune the model, and develop secure code patches. Listing 1 demonstrates the syntax for prompt engineering used to fine-tune the *CodeT5* and generate the fixed version of the vulnerable code. The <*vulnerable code block*> includes the exact snippet of code that contains the potential vulnerability. The <*fix version of the vulnerable code*> is the code block that the model is expected to output the corrected version of the provided code snippet by any necessary modifications to mitigate the vulnerabilities identified. Similarly, Listing 2 presents the syntax of the patch generation prompt used to query and fine-tune the *CodeLlama* model.

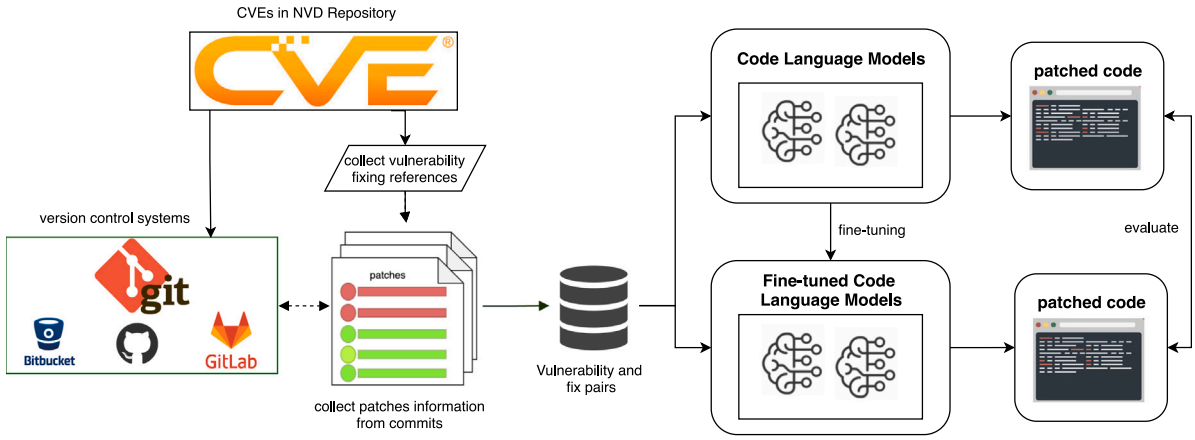

Fig. 1. The proposed framework for the automated patch generation fine-tuning the CLM on vulnerable-fix pairs of the source code.

```
Vulnerable program code of the <programming
    language> programming language:

<vulnerable code block>

Patch of the program is:

<fix version of the vulnerable code>
```

Listing 1: The syntax of the patch generation prompt used to fine-tune the CodeT5 model and produce the fixes

```
You are a powerful code-fixing model.
Your job is to analyze and fix
    vulnerabilities in code.
You are given a snippet of vulnerable code.
You must output the fixed version of the
    code snippet.

### Input:
What is the fixed version of the code for
    the following vulnerability?

### Vulnerable:
<vulnerable code block>

### Response:
<fix version of the vulnerable code>
```

Listing 2: The syntax of the patch generation prompt used to fine-tune the CodeLlama model and produce the fixes

The Listing 3 shows a simple example demonstrating a buffer overflow vulnerability in C and its fixed code version. The vulnerable code uses *strcpy* (CWE-120), which does not check the length of the destination buffer, leading to a buffer overflow. Where the fixed code uses *strncpy*, which limits the number of characters copied to prevent overflow and ensures the buffer is null-terminated.

```
You are a powerful code-fixing model.
Your job is to analyze and fix
    vulnerabilities in code.
You are given a snippet of vulnerable code.

You must output the fixed version of the
    code snippet.

### Input:
What is the fixed version of the code for
    the following vulnerability?
```

```
### Vulnerable:
char buffer[10];
strcpy(buffer, "very long string");


### Response:
char buffer[10];
strncpy(buffer, "very long string",
    sizeof(buffer) - 1);
buffer[sizeof(buffer) - 1] = '\0';  //
    Ensure null-termination
```

Listing 3: An example of the patch generation prompt showing a simple example demonstrating a buffer overflow vulnerability in C and its fix

The choice of prompts represents a structured approach to model fine-tuning and evaluation. For fine-tuning the *CodeT5* and *CodeLlama* models in secure code generation, the prompts are designed with simplicity, task orientation, and adherence to secure coding principles. They are direct and concise to ensure clarity and task understanding. Task-oriented prompts explicitly guide the models in analyzing and addressing vulnerabilities. Furthermore, the prompts are grounded in real-world applicability, focusing on practical scenarios to ensure their relevance to common vulnerabilities.

3.2. Experimental setup

The resource-intensive tasks, precisely fine-tuning the CLM and patch generation, were executed on a high-performance computing (HPC) infrastructure: NVIDIA DGX. The system has dual Intel Xeon Scalable Platinum 8176 processors, and 8 NVIDIA Volta V100 GPU (graphical processing unit). Moreover, it is designed for parallel executions, making it ideal for the matrix multiplications required in neural network training. We used a general-purpose PC for dataset extraction, powered by an AMD Ryzen 7 5800H processor running at 3.2 GHz, 16 GB of RAM, and an NVIDIA RTX 3080 GPU with 16 GB of memory. Downloading and processing the data on the machine took approximately two hours for *FixMe* vulnerability dataset.

3.3. Evaluation environment

The machine learning experiments and evaluations were conducted in a well-defined environment to ensure reproducibility and consistency of results. Table 3 shows a list of required dependencies and their versions used in the research environment. The setup includes CUDA version 12.1.1 for GPU acceleration and Python 3.10.0 as the primary programming language. Key libraries utilized for model training and

Table 3
System configuration of the experimental environment.

Component	Version
CUDA	12.1.1
Python	3.10.0
Accelerate	0.32.1
Codebleu	0.7.0
Datasets	2.20.0
Pandas	2.2.3
Peft	0.12.0
PyYAML	6.0.2
Torch	2.4.0
Transformers	4.43.3

evaluation include Transformers 4.43.3 for handling pre-trained language models, Torch 2.4.0 for deep learning computations, and datasets 2.20.0 for managing data processing pipelines. PyYAML 6.0.2 facilitated configuration management. This configuration ensures that all necessary components are compatible and optimized for the machine-learning tasks performed in this study.

3.4. Dataset information

In this study, we have experimented with two datasets, primarily *FixMe* [50] and *RepairLlama* [51] datasets. Both datasets offer valuable information for the CLM research community working on automated program repair tasks.

This study used a recently released version of the *FixMe* dataset. The dataset is a collection of code patches from the commits of various version control systems, which covers a wide array of programming languages. Moreover, the security vulnerability-fix pairs on the dataset are based on the Common Vulnerabilities and Exposures (CVE) records, and labeled to Common Weakness Enumeration (CWE) types. The dataset covers various features associated with patch hunks, the name of the affected file, an identifier of the hunk, patch information, and source and target code lines before and after applying the patch. The data also includes the counts of lines added and removed, code snippets before (vulnerable) and after (patch or fix), start lines and lengths of the source and target code affected, section header for the hunk, size of the hunk, and programming language of the code involved. In this study, we have used a few features of the patch hunk data, i.e., code snippets before (vulnerable) and after (patch or fix), and programming_language of it. There are, in total, 1495 vulnerability-fix pairs at hunk level data in the *FixMe* dataset that we have considered in the study.

The *RepairLlama* dataset is a valuable resource designed to advance automated program repair tasks by offering processed fine-tuning datasets. Researchers can explore and load specific revisions, which are input/output representation pairs, using the Python datasets library, with examples such as *ir1xor1* for basic code changes and *irXxorY* for more complex transformations. This flexibility allows for focused fine-tuning of language models on various repair scenarios. The dataset version we considered in this study includes approximately 65,000 vulnerability-fix pairs.

4. Results

We assess the capabilities of two transformer-based baseline models, *CodeT5* and *CodeLlama*, in generating patches for vulnerable code. This section provides a comprehensive analysis of the performance metrics and a comparison between baseline models and fine-tuned instruct models across various datasets.

CodeT5 is a new pre-trained encoder–decoder model for programming languages, which is pre-trained on 8.35M functions in 8 programming languages (Python, Java, JavaScript, PHP, Ruby, Go, C, and C#) [21]. We included C++ as well because its vulnerabilities are similar to those found in the C programming language. The *CodeLlama* model we experimented with in this study is *codellama/CodeLlama-7b-Instruct-hf*, a 7B instruct-tuned version available in the Hugging Face [49]. The model is optimized for general code synthesis and comprehension. Training a large model, even one with 7B parameters, requires significant computational resources and numerous GPUs. Due to our resource constraints and to reduce the execution time for fine-tuning, we have converted the original *CodeLlama-7B-Instruct-hf* model into a 4-bit quantized form and used this optimized version of the model. We considered 20 epochs for fine-tuning the *CodeLlama* model. Table 4 summarizes the configuration settings and hyperparameters for the models for optimal model performance for the patch generation task. Moreover, the parameters control the model's behavior during the fine-tuning process.

Table 4
Configuration settings and hyperparameters for fine-tuning the models for security patch generation task.

Configuration	Value
Max new tokens	512
Temperature	2.0
Num beams	1
Tokenizer	Roberta
Learning rate	1e−5
Num train epochs	100
Weight decay	0.01
Logging steps	100
Max steps	100
Max hunks per URL	1

4.1. Performance scores

To evaluate the performance of a patch generation model for vulnerable source code blocks, CodeBLEU and ROUGE scores can be computed to compare the CLMs. Several similar studies have also carried out evaluation of transformer models with the help of BLEU, CodeBLEU, ROUGE scores. The translation model, PATCHEXPLAINER [52], leverages BLEU scores to evaluate the patch description similarity. The PATCHEXPLAINER is trained to generate patch descriptions as a machine translation task. Moreover, VulMaster [40], a Transformer-based neural network model, has also used the CodeBLEU score to evaluate the generated vulnerability repairs by comprehensively comparing them with the actual vulnerable code.

4.1.1. CodeBLEU score

BLEU is a metric commonly used to evaluate the quality of machine-translated text against one or more reference translations. Specifically, it measures how closely a candidate translation matches the reference translations by calculating *n-gram precision*. Additionally, it applies a *brevity penalty* to penalize overly short translations, ensuring that translations are both precise and complete. The BLEU does not consider semantic similarity and contextual understanding of the code. Therefore, CodeBLEU [53] is an extension of the BLEU score designed to evaluate code. CodeBLEU provides a more comprehensive evaluation of generated code by incorporating syntax, semantics, and functional behavior, making it a better fit for programming tasks than BLEU. The measure includes the following weighted BLEU and additional features; syntax matching, data flow, and F-measure.

Weighted BLEU: Similar to BLEU, it calculates n-gram precision but weights it to account for differences in programming languages. Moreover, the BLEU score ranges from 0 to 1 (or 0% to 100%), where a higher score indicates a closer match to the reference translation. The *BLEU-like Score* computes the geometric mean of the n-gram precision scores (weighted by w_n), exponentiates it, and finally multiplies it by the brevity penalty (BP). The weights w_n in the BLEU-like Score determine the relative importance of different n-gram precisions in the final metric. Typically, uniform weights ($w_n = \frac{1}{N}$) are used, ensuring

equal contributions from unigrams, bigrams, trigrams, and higher-order n-grams. The BP penalizes translations that are too short, ensuring that a model does not simply generate short phrases to artificially boost precision, calculated as in Eq. (2):

$$BP = \begin{cases} 1, & \text{if } c > r \\ \exp\left(1 - \frac{r}{c}\right), & \text{if } c \leq r \end{cases} \tag{2}$$

where,

- c is the length of the generated/candidate code.
- r is the length of the reference code.

The *BLEU-like Score* is formally defined by Eq. (3).

$$\text{BLEU-like Score} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{3}$$

where,

- p_n: precision for n-grams,
- w_n: weights assigned to different n-grams, and
- BP: Brevity Penalty as defined in Eq. (2).

AST similarity: Considers syntax correctness by analyzing the AST of the code. The value ranges from 0 to 1 (or 0 to 100%), where 1 indicates identical syntax structure (Eq. (4)). AST is a tree representation of the abstract syntactical structure of source code, each tree node denotes a construct occurring in the source code. In essence, ASTs help represent the code's structure, as they can ignore aspects like parentheses or certain statements that can be inferred from the context. The ASTs are used in CodeBLEU to capture more semantic and syntactic details than n-gram overlap can provide.

$$\text{AST Similarity} = \min\left(1, \frac{\text{Number of the matched nodes}}{\text{Total nodes in reference AST}}\right) \tag{4}$$

where *matched nodes* refer to the structural components in the AST of the candidate code that align with the reference AST, which represents syntactic constructs like loops, conditionals, and function calls.

Data-flow similarity: checks how data flows between variables in the code to verify correctness. The value ranges from 0 to 1 (or 0 to 100%), where 1 indicates an identical data flow graph (DFG) as shown in Eq. (5). Moreover, data flow analysis is a method used in compiler construction to gather information about the possible set of values calculated at various points in a computer program. It refers to the sequence and processes that data follows within a system, mainly used for optimization. In CodeBLEU, data flow analysis allows the measurement of how data changes and moves through the code, effectively capturing the program's semantics beyond the scope of simple text overlap metrics.

$$\text{Data-flow Sim.} = \min\left(1, \frac{\text{Number of the matched edges}}{\text{Total edges in reference DFG}}\right) \tag{5}$$

where, *matched edges* refer to data dependencies (relations between nodes) that exist in both the reference and generated data-flow graphs.

Token matching (F-measure): Combines precision and recall for token-level matching in the code. Similarly, the f-measure ranges from 0 to 1 (or 0 to 100%), where 1 indicates perfect precision and recall.

$$Precision = \frac{\sum_{n=1}^{N} min(C_{hyp}, C_{ref})}{\sum C_{hyp}} \tag{6}$$

$$Recall = \frac{\sum_{n=1}^{N} min(C_{hyp}, C_{ref})}{\sum C_{ref}} \tag{7}$$

where,

- C_{hyp}: count of n-grams in the hypothesis (candidate code).
- C_{ref}: count of n-grams in the reference code.
- $\min(C_{\text{hyp}}, C_{\text{ref}})$: number of matched n-grams between the hypothesis and reference.

Similarly, the F-measure, which combines precision (Eq. (6)) and recall (Eq. (7)), is given by Eq. (8):

$$F_{measure} = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{8}$$

CodeBLEU: The final CodeBLEU score (Eq. (9)) is calculated as the sum of the weighted BLEU score (Eq. (3)), F-measure (Eq. (8)), AST similarity (Eq. (4)), and dataflow similarity (Eq. (5)). The calculated CodeBLEU score ranges from 0 to 1 (or 0% to 100%):

$$\text{CodeBLEU} = \alpha \times \text{Weighted BLEU} + \delta \times F_{\text{measure}} + \beta \times \text{AST} + \gamma \times \text{DataFlow} \tag{9}$$

where, α, δ, β, and γ are predefined weights (typically set to 0.25 each).

4.1.2. ROUGE score

Recall-Oriented Understudy for Gisting Evaluation (ROUGE) [54] is a set of metrics for evaluating automatic summarization and machine translation, including patch code generation. It includes measures to automatically determine the quality of a summary by comparing it to other (ideal) summaries created by humans. The measures count the number of overlapping units, such as n-grams, word sequences, and word pairs, between the computer-generated summary (the patch in the code generation context) and ideal summaries (the reference code). The range for ROUGE-N Recall, ROUGE-N Precision, and ROUGE-N F1 Score is 0 to 1 (or 0% to 100%). Technically, scores closer to 1 indicate high similarity between the generated patch and the reference, while scores closer to 0 indicate little to no similarity. ROUGE scores are calculated using n-grams, which include unigrams (ROUGE-1), bigrams (ROUGE-2), and L-grams (ROUGE-L). The ROUGE-L, specifically, is based on the longest common subsequence (LCS) between the reference and the hypothesis. The LCS captures fluency and coherence by accounting for the order of tokens. ROUGE scores are formally defined as follows.

ROUGE-N Recall: This is the proportion of n-grams in the reference also present in the generated patch. Mathematically, for n-grams, this can be represented as in Eq. (10):

$$RougeN_{recall} = \frac{\text{Number of matching } n\text{-grams}}{\text{Total no. of } N\text{-grams in the reference}} \tag{10}$$

ROUGE-N Precision: This is the proportion of n-grams in the generated patch that are also present in the reference as follows (Eq. (11)):

$$RougeN_{precision} = \frac{\text{Number of matching } n\text{-grams}}{\text{Total no. of } N\text{-grams in the hypothesis}} \tag{11}$$

ROUGE-N F1 score: The harmonic mean of precision and recall is defined as follows in Eq. (12), which is based on Eqs. (10)–(11):

$$RougeN_{F1} = \frac{2 \cdot RougeN_{recall} \cdot RougeN_{precision}}{RougeN_{recall} + RougeN_{precision}} \tag{12}$$

RougeLsum score: RougeLsum [55] is a variant of the ROUGE metric designed to provide a more comprehensive evaluation of text summarization quality. RougeLsum combines Rouge-1, Rouge-2, and Rouge-L aggregating the scores, as presented in Eq. (13) which is based on Eq. (12).

$$\text{RougeLsum} = \alpha_1 \cdot \text{Rouge-1} + \alpha_2 \cdot \text{Rouge-2} + \alpha_3 \cdot \text{Rouge-L} \tag{13}$$

where,

- Rouge-1 (or $Rouge1_{F1}$): F1 score for unigram overlap.
- Rouge-2 (or $Rouge2_{F1}$): F1 score for bigram overlap.
- Rouge-L (or $RougeL_{F1}$): F1 score for LCS-based similarity.
- $\alpha_1, \alpha_2, \alpha_3$: weights assigned to Rouge-1, Rouge-2, and Rouge-L. The default value of α is used for ROUGE F-score for equal weighting of precision as specified in the *evaluate* library [55].

Table 5
Comparison of the fine-tuned model performance with the base model on vulnerable-fix pairs corresponds with multiple programming languages (only single hunk commits).

Type	Score	Base	Instruct	Improvement
Codebleu	codebleu_score	25.12%	34.06%	8.94%
	ngram_match_score	0.43%	10.39%	9.97%
	weighted_ngram_match_score	0.66%	14.98%	14.32%
	syntax_match_score	25.26%	43.97%	18.72%
	dataflow_match_score	22.79%	31.55%	8.76%
Rouge	rouge1	16.98%	32.66%	15.69%
	rouge2	8.88%	23.56%	14.69%
	rougeL	15.48%	29.70%	14.22%
	rougeLsum	16.95%	32.30%	15.35%

4.2. Performance comparison with CodeT5 based instruct models

The effectiveness of our approach is evaluated using enhanced versions of CodeBLEU and ROUGE scoring metrics. The result indicates substantial improvements in both the CodeBLEU scores, which capture the model's proficiency in generating patches, and the ROUGE scores, which show the model's ability to retain the original program's logic. The following points show the performance of the multiple instruct fine-tuned models generated on various variants of the datasets.

4.2.1. Model 1: Fine-tune on multiple programming languages data (only single hunk commits) of fixme

To generate an in-context fine-tuned model to create patches of different programming languages that *CodeT5* supports, we have included the filtered version of the entire *FixMe* dataset that only consists of the following programming languages: C, C++, Python, Java, JavaScript, PHP, Ruby, Go, and C#. The dataset comprised 1494 entries with columns for *code_before*, *code_after*, and *programming_language*. Then, the data was split into training (80%), validation (10%), and test (10%) sets, with shapes of 1195, 149, and 150 entries, respectively.

Table 5 compares the performance measures of the *CodeT5* base model with the fine-tuned model on vulnerable-fix pairs corresponding with multiple programming languages considering only single hunk commit changes of *FixMe* dataset. The fine-tuned model outperforms the base model across all performance metrics, including 8.94% increase in overall CodeBLEU (from 25.12% to 34.06%), with notable improvements in syntax match (18.72%) and weighted n-gram match (14.32%). ROUGE scores also show substantial gains, with ROUGE-1 improving by 15.69%, ROUGE-2 by 14.69%, and ROUGE-Lsum by 15.35%, reflecting better text generation capabilities. These results highlight the effectiveness of fine-tuning *CodeT5* for handling single-hunk commit changes in multi-language code vulnerability repair.

4.2.2. Model 2: Fine-tune on Java code blocks (only single hunk commits) of fixme dataset

We have fine-tuned the *CodeT5* model on vulnerable-fix code blocks (486 pairs) of Java programming language with only single hunk commits to create an instruct model targeted for Java security fix generation task. Table 6 and Fig. 2 present the comparison of scores between a base model and an instruct model across different metrics for evaluating patch generation. The scores are categorized into two main types: CodeBLEU and ROUGE. Each type has multiple sub-metrics, showing the performance of the models and the improvement achieved by the instruct model over the base model. The result demonstrates that the fine-tuned instruct model improves the CodeBLEU and ROUGE metrics better than the baseline *CodeT5* model. Significant improvements are observed in syntax_match_score (27.74%) and dataflow_match_score (46.35%).

Table 6
Weighted average BLEU scores and ROUGE scores improvement on fine-tuned instruct model on Java programming language data (only single hunk commits) comparing performance measures.

Type	Score	Base	Instruct	Improvement
Codebleu	codebleu	26.17%	32.50%	6.33%
	ngram_match_score	0.37%	11.73%	11.36%
	weighted_ngram_match_score	0.35%	15.23%	14.89%
	syntax_match_score	15.20%	42.95%	27.74%
	dataflow_match_score	1.25%	47.60%	46.35%
Rouge	rouge1	10.88%	32.62%	21.74%
	rouge2	6.04%	23.32%	17.28%
	rougeL	10.88%	31.11%	20.23%
	rougeLsum	10.88%	32.44%	21.55%

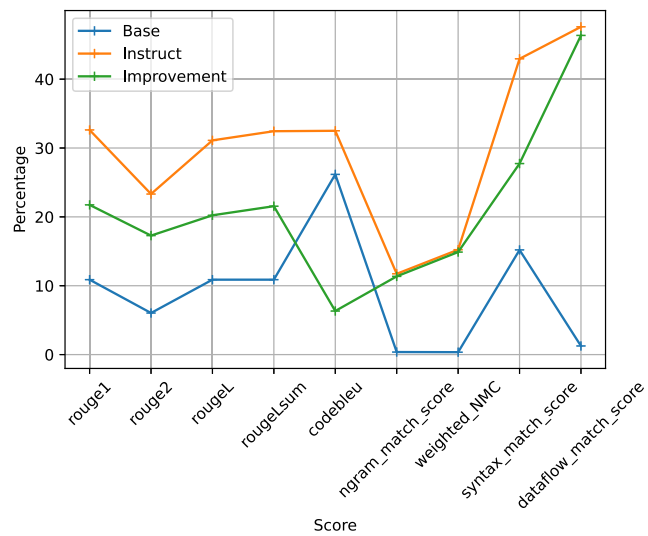

Fig. 2. Fine-tuned instruct model on data with Java programming language (only single hunk commits) comparing the performances with base model.

Table 7
Weighted average BLEU scores and ROUGE scores improvement comparing base and instruct model fine-tuning on C/C++ (single hunk commits) of *FixMe* with *CodeT5*.

Type	Score	Base	Instruct	Improvement
Codebleu	codebleu	20.48%	33.88%	13.40%
	ngram_match_score	0.79%	12.79%	12.00%
	weighted_ngram_match_score	1.12%	20.28%	19.16%
	syntax_match_score	16.02%	40.93%	24.91%
	dataflow_match_score	31.45%	42.23%	10.78%
Rouge	rouge1	23.17%	39.11%	15.94%
	rouge2	12.79%	29.75%	16.96%
	rougeL	20.97%	34.97%	14.00%
	rougeLsum	22.95%	38.68%	15.74%

4.2.3. Model 3: fine-tune on C/C++ (single hunk commits) data of fixme dataset

To construct an instruct model targeted for the patch generation task for C/C++, we fine-tuned the *CodeT5* base model only on the C/C++ vulnerability-fix pairs (1533 vulnerability-fix pairs) of *FixMe* dataset. Table 7 compares the performance of text and code generation using base *CodeT5* and fine-tuned instruct models, highlighting significant improvements across various metrics. For ROUGE scores, ROUGE-1 improved from 23.17% to 39.11%, ROUGE-2 from 12.79% to 29.75%, ROUGE-L from 20.97% to 34.97%, and ROUGE-Lsum from 22.95% to 38.68%. Similarly, CodeBLEU scores also show notable enhancements; CodeBLEU scores increased from 20.48% to 33.88%. These results indicate that the instruct model significantly outperforms the base model, leading to better performance in both patch versions of the patch generation task.

Table 8
Weighted average BLEU scores and ROUGE scores improvement comparing base *CodeT5* and instruct model fine-tuning on *RepairLlama* dataset.

Type	Score	Base	Instruct	Improvement
Codebleu	codebleu	14.09%	32.32%	18.23%
	ngram_match_score	0.81%	14.14%	13.33%
	weighted_ngram_match_score	2.12%	23.24%	21.12%
	syntax_match_score	18.02%	43.74%	25.72%
	dataflow_match_score	35.42%	48.18%	12.76%
Rouge	rouge1	26.18%	43.16%	16.98%
	rouge2	18.67%	47.57%	28.90%
	rougeL	24.46%	41.49%	17.03%
	rougeLsum	23.10%	44.07%	20.97%

Table 9
Weighted average BLEU scores and ROUGE scores improvement: Fine-tuned CodeLlama-based instruct model on *FixMe* data with Java programming language.

Type	Score	Base	Instruct	Improvement
Codebleu	codebleu	52.42%	67.04%	14.63%
	ngram_match_score	19.08%	47.17%	28.09%
	weighted_ngram_match_score	70.55%	82.17%	11.62%
	syntax_match_score	65.37%	71.41%	6.04%
	dataflow_match_score	63.64%	68.93%	5.29%
Rouge	rouge1	43.68%	49.14%	5.46%
	rouge2	35.40%	41.38%	5.98%
	rougeL	40.94%	46.48%	5.54%
	rougeLsum	40%	45.67%	5.67%

Table 10
Weighted average BLEU scores and ROUGE scores improvement by fine-tuned *CodeLlama*-based instruct model on *FixMe* data with multiple programming languages.

Type	Score	Base	Instruct	Improvement
Codebleu	codebleu	54.35%	63.46%	9.12%
	ngram_match_score	17.26%	25.42%	8.16%
	weighted_ngram_match_score	70.45%	83.23%	2.78%
	syntax_match_score	73.51%	79.67%	2.16%
	dataflow_match_score	63.40%	74.22%	5.82%
Rouge	rouge1	49.73%	59.24%	9.51%
	rouge2	41.96%	41.18%	−0.78%
	rougeL	46.93%	57.35%	10.42%
	rougeLsum	46.21%	52.64%	6.43%

Table 11
Weighted average BLEU scores and ROUGE scores improvement by fine-tuned *CodeLlama*-based instruct model on *RepairLlama*.

Type	Score	Base	Instruct	Improvement
Codebleu	codebleu	54.48%	60.13%	5.67%
	ngram_match_score	24.26%	35.15%	10.89%
	weighted_ngram_match_score	63.45%	68.28%	4.83%
	syntax_match_score	74.16%	75.42%	1.26%
	dataflow_match_score	61.96%	67.78%	5.82%
Rouge	rouge1	47.72%	58.14%	10.42%
	rouge2	43.93%	57.33%	13.40%
	rougeL	47.27%	58.34%	11.07%
	rougeLsum	46.32%	57.94%	11.62%

4.2.4. Model 4: CodeT5 instruct model on RepairLlama dataset

The instruct model, fine-tuned on the *RepairLlama* dataset (~65,000 vulnerability-fix pairs), significantly outperforms the base *CodeT5* model across all evaluated metrics, showcasing notable improvements in the patch generation task. Table 8 presents the performance comparison between the base *CodeT5* model and its instruct version fine-tuned on the *RepairLlama* dataset. The CodeBLEU scores highlight substantial gains, particularly in syntax and ngram matching, with syntax match showing the highest improvement of 25.72%. The instruct model also excels in ROUGE metrics, specially with rouge2 demonstrating the most significant improvement of 28.90%, indicating enhanced sequence matching capabilities. The aggregated rougeLsum score has been improved by approximately 21%. These results underscore the instruct model's improved ability to generate more accurate, context-aware outputs, making it a superior choice for code and language tasks compared to the base model.

4.3. Performance comparison with CodeLlama based instruct models

The following subsections present the multiple instruct models generated on multiple variants of the *FixMe* and *RepairLlama* datasets. The results demonstrate considerable improvements in both performance scores, reflecting the model's enhanced capability to generate accurate and coherent patches. Similar to the earlier subsection, we computed CodeBLUE and ROUGE scores to compare the patch generation performance with base and instruct models.

4.3.1. Model 5: Fine-tuning on java code blocks of FixMe dataset

This particular instruct model was fine-tuned on Java vulnerability-fix data (486 pairs) of *FixMe*, only considering the single hunk commits. The instruct model significantly outperforms the base *CodeLlama* model as in *CodeT5* model, across all evaluated metrics, showcasing notable improvements in the patch generation task. The CodeBLEU improved by more than 14.63%, and ROUGE scores improved by more than 5%, which highlights substantial enhancements. Table 9 presents the performance comparison between the base *CodeLlama* model and its instruct version fine-tuned on the *FixMe*-Java dataset.

4.3.2. Model 6: Fine tuning on multiple programming languages code blocks of fixme dataset

The instruct model fined-tuned on all the supported programming languages of the single hunk data of *FixMe* dataset (same size as in Model 1) significantly outperforms the base *CodeLlama* model across all evaluated metrics except rouge2, showcasing notable improvements in the patch generation task. The CodeBLEU (by > 9%)and aggregated ROUGE score (rougeLsum > 6%) highlight substantial improvements. Table 10 presents the performance comparison between the base *CodeLlama* model and its instruct version fine-tuned on the *FixMe* multi-programming languages (C, C++, Java, Python, JavaScript, PHP, Ruby, Go, and C#) dataset.

4.3.3. Model 7: CodeLlama instruct model on RepairLlama dataset

The instruct model based on fine-tuned *CodeLlama* on the *RepairLlama* dataset shows significant performance improvements over the base model, as evidenced by enhanced CodeBLEU and ROUGE scores. The dataset includes ~65,000 vulnerability-fix pairs. The CodeBLEU score improved by 5.67%, reflecting a substantial enhancement in overall code quality, with notable gains in n-gram match (10.89%) and dataflow match (5.82%), indicating better sequence alignment and data coherence. Although the syntax match score saw a modest increase of 1.26%, it still highlights improved structural accuracy. ROUGE scores also demonstrated considerable gains, with ROUGE-2 showing the highest improvement of 13.40%, underscoring the model's ability to retain program logic and sequence accuracy. These results collectively highlight the instruct model's superior capability in generating more precise and contextually relevant fixes compared to the base *CodeLlama* model (see Table 11).

4.4. Training and inference time

The fine-tuning and inference durations can fluctuate based on the available computational resources, such as the type and number of GPUs. Table 12 compares the fine-tuning and inference time of two models, *CodeT5* and *CodeLlama*, evaluated on two different datasets: *FixMe* and *RepairLlama*. The reported time span reflects the duration needed to fine-tune the models using all possible vulnerable code pairs across all supported programming languages. These results highlight

Table 12
Training and inference time for *CodeT5* and *CodeLlama* models on *FixMe* and *RepairLlama* datasets.

Model	Dataset	Fine-tuning time	Inference
CodeT5	*FixMe*	13 h	3 s
	RepairLlama	26 h	5 s
CodeLlama	*FixMe*	23 h	5 s
	RepairLlama	43 h	6 s

that while *CodeLlama* demonstrates enhanced performance, as seen in previous evaluations, it requires significantly more time for both training and inference compared to *CodeT5*, emphasizing a trade-off between performance gains and computational costs.

We have made the source code, configuration settings, and fine-tuned instruct models publicly available to ensure the reproducibility of the results presented in our study. Future researchers can build upon these results, further refine them, or incorporate other models for comparison and analysis. In the following section, we have further discussed the applications and limitations of the APR using CLM.

5. Discussion

Our work stands out from previous code representations in several key aspects. First, we develop code representations that align closely with the pre-training data and objectives, allowing PatchLM to effectively leverage pre-learned knowledge. Second, our approach supports multiple programming languages and various types of vulnerabilities, addressing a critical challenge in program repair. Furthermore, the data we use is derived from real-world vulnerability and fix pairs associated with CVE records, rather than relying on synthetic datasets that may lack practical relevance. In this section, we explain the limitations of CLMs, specially those designed for automated patch generation tasks, and discuss several threats to the validity of the study.

5.1. Limitations of CLM for automated patch generation

The use of CLM for automated patch generation in software security has shown promising potential, but it also comes with several limitations. These limitations can impact the effectiveness, accuracy, and reliability of the patches generated. Some of the fundamental limitations are discussed as follows.

Lack of vulnerability-fix pairs: Most of the vulnerability datasets are designed for classification tasks rather than for generating automated patches or fixes. This presents a significant challenge for software security, especially in the context of APR systems, due to the scarcity of data that pairs vulnerabilities with their corresponding fixes [16,56,57]. Fixing a vulnerability often involves adding, removing, or modifying existing code, which may occur at different levels of granularity. For instance, a developer might introduce a new function or file to address a vulnerability, meaning the repair may not directly modify the existing vulnerable code. As a result, even when the same vulnerability-fix pair is collected, the fix might not be applied to the precise code segment where the vulnerability was initially identified.

Understanding context and semantics: CLM has demonstrated promising performance in understanding code syntax but struggles with comprehending code semantics, especially dynamic semantics. Ma et al. [8] have concluded that LLMs exhibit capabilities similar to an AST parser, showing initial competence in static code analysis. However, the output of LLMs is prone to hallucinations when interpreting the semantic structure of code and fabricating nonexistent facts.

Accuracy and reliability: Despite significant advancements in the field, various approaches to solving the problem of automated patch generation using CLM continue to yield low accuracy [14,58]. One of the best approaches to evaluate the generated code is to check its compilation and execution success, ensuring it is free from vulnerabilities. Therefore, the LLM/CLM should generate logically correct code and accurately measure the output.

Lack of security-specific knowledge: The identified patterns and detected vulnerabilities are based on specific experimental datasets, which may limit their applicability to other software, such as closed-source projects or specialized security libraries [56,59,60]. To improve the effectiveness of fine-tuned APR models, it is crucial for developers to have a deep understanding of the domain and hands-on experience in generating actual vulnerability-fix data. This practical knowledge would help refine the models, making them more reliable and capable of addressing a broader range of programs.

Lack of argumentation of the samples: A significant limitation in the current approach is the lack of diversity and argumentation in the sample datasets used for training and testing. When the samples are not adequately varied or do not cover a wide range of real-world scenarios, the models may fail to capture the complexities and nuances of different types of vulnerabilities [61]. This can lead to a narrow understanding and reduce the generalizability of the models. To improve their robustness, it is essential to incorporate a broader set of samples that reflect diverse coding practices, environments, and potential vulnerabilities. By doing so, the models can better understand and address a more comprehensive array of security issues in different contexts [62,63].

Lack of evaluation criteria: One of the severe flaws in the current studies is the use of unsuitable criteria for evaluating the performance of automated repair techniques [14]. Accuracy measure cannot distinguish whether the output of a model is close to being correct or completely wrong. Software code evaluation measures such as BLEU and CodeBLEU [53] have gained more attention for the code translation task. Several existing studies [21,22,28,34] have utilized the CodeBLEU for code translation. Contrary to the studies, Jiang et al. [64] have noticed that BLEU and CodeBLEU are misleading in comparing APR techniques because CLMs generate more diverse patches, requiring better automated metrics.

5.2. Threats to validity

Assessing the performance of code repair models requires a careful and systematic approach to ensure the reliability and applicability of the findings. While PatchLM demonstrates promising capabilities in addressing vulnerabilities, the complexity of software systems and the nuances of code repair pose challenges that can introduce potential threats to validity. These challenges stem from various factors, including the design of datasets, the choice of evaluation metrics, and the generalizability of the results to real-world scenarios.

Internal validity:

The design of the dataset and the choice of evaluation metrics play a crucial role in assessing the performance of models. The employed metrics, such as ROUGE and CodeBLEU, while widely used, have notable limitations in the context of code repair. These metrics focus mainly on measuring textual similarity rather than evaluating functional correctness or improvements in security—both of which are critical when addressing vulnerabilities [41]. Other functional correctness metrics, such as execution-based evaluation (Pass@k), can be utilized to assess the proportion of generated solutions that successfully pass predefined test cases, offering a more accurate measure of the code's functional correctness [60]. This makes them particularly relevant for tasks like code repair and generation. Additionally, incorporating human evaluation can provide valuable insights by verifying whether the repaired code is functionally correct and adheres to practical coding standards.

External validity: In this study, the proposed method focuses on utilizing code changes made by the commit in the same file name as the before-code and after-code. One potential threat is related to the granularity level of the fine-tuned data. Fixing security vulnerabilities may involve multiple functions, classes, or files. Fixing one function might necessitate changes in other dependent functions or classes.

Some vulnerabilities involve configuration files or settings that influence multiple modules. Secondly, one potential threat to the validity of our study is the possibility of selection bias, as we have focused exclusively on *CodeT5* and *CodeLlama* models. A wide range of pre-trained CLMs is publicly available online. However, the models we selected for fine-tuning are well-established and robust representatives, pre-trained on source code. Additionally, we applied quantization to the original *CodeLlama* model to fine-tune it using vulnerability-fix data pairs. Exploring the original non-quantized model (32-bit) or applying higher levels of quantization could potentially enhance the performance of the fine-tuned model.

5.3. Directions for future work

Building on the current study, several directions for future work can be explored to strengthen the effectiveness and generalizability of automated code repair models. First, to improve the robustness and real-world applicability of APR models, future research should prioritize more comprehensive evaluation strategies. Moving beyond ROUGE and CodeBLEU metrics, it is essential to incorporate further metrics like METEOR score, BERTScore, BARTScore [65], and execution-based metrics like Pass@k [66], which better capture functional correctness. Additionally, integrating automated security analysis tools and human evaluations can provide deeper insights into the effectiveness and reliability of generated patches. Second, the development of bigger and more granular datasets encompassing multi-file changes, configuration updates, and broader contextual information will also be crucial in training models capable of addressing the complexities of real-world vulnerabilities. Moreover, security-focused fine-tuning using vulnerability-specific datasets, such as those based on CWE classifications or CVE-reported flaws, can enhance the model's ability to detect and repair real-world security-critical issues. Third, further work should explore a more diverse set of pre-trained CLMs, for example, *Qwen2.5-Coder-32B-Instruct [67], deepseek-coder-33b-instruct [68]*, bigger *CodeLlama-70b-Instruct-hf [69]*, assessing both full-precision and quantized versions to understand their trade-offs in performance and efficiency. Integrating these models into real-world development environments, such as CI/CD pipelines, and collecting feedback from developers will help evaluate their practical utility and inform future design choices.

6. Conclusion

In conclusion, this paper proposed PatchLM model designed to enhance automated program repair by fine-tuning generic CLMs. The proposed model is capable of generating patches for given vulnerable code snippets. By leveraging this model, developers can significantly reduce debugging costs and improve software quality by automatically generating patches to rectify vulnerabilities. Additionally, the ongoing fine-tuning and utilization of an expanded dataset offer promising potential for further advancements in vulnerability fixing and overall software security.

CRediT authorship contribution statement

Guru Bhandari: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Data curation, Conceptualization. **Nikola Gavric:** Writing – review & editing, Resources, Methodology, Conceptualization. **Andrii Shalaginov:** Writing – review & editing, Supervision, Project administration, Conceptualization.

Compliance with ethical standards

This material is the author's original work, which has not been previously published elsewhere. The paper reflects the author's research and analysis wholly and truthfully.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The data extraction process received substantial support from the Kristiania-HPC infrastructure hosted at Kristiania University of Applied Sciences. Additionally, the machine learning experiments in this research study were made possible by the eX3 HPC infrastructure hosted at Simula Research Laboratory.

Data availability

The source code of the study is publicly available at the GitHub repository- https://github.com/SmartSecLab/PatchLM. The experimented *FixMe* dataset is available at https://zenodo.org/records/10955342 and *RepairLlama* at https://huggingface.co/datasets/ASSERT-KTH/repairllama-datasets. We welcome the software security community to reproduce our results and further enhance the fixing methods for software vulnerabilities and weaknesses.

References

[1] NIST, National vulnerability database (NVD), 2024, URL https://nvd.nist.gov/. (Accessed 16 December 2024).
[2] VulnCheck, Danger is still lurking in the NVD backlog, 2024, URL https://vulncheck.com/blog/nvd-backlog-exploitation-lurking. (Accessed 16 December 2024).
[3] Q.-C. Bui, R. Paramitha, D.-L. Vu, F. Massacci, R. Scandariato, APR4vul: an empirical study of automatic program repair techniques on real-world Java vulnerabilities, Empir. Softw. Eng. 29 (1) (2023) 18, http://dx.doi.org/10.1007/s10664-023-10415-7.
[4] K.D. Pramod, W.T.N. De Silva, W.U.K. Thabrew, R. Shariffdeen, S. Wickramanayake, BUGSPHP: A dataset for automated program repair in PHP, in: Mining Software Repositories 2024, IEEE Computer Society, 2024, pp. 128–132, http://dx.doi.org/10.1145/3643991.36448, arXiv:2401.07356 [cs].
[5] GitHub, GitHub copilot · your AI pair programmer, 2024, URL https://github.com/features/copilot. (Accessed 24 June 2024).
[6] AlphaCode, Competitive programming with AlphaCode, 2022.
[7] M. AI, Introducing code llama, a state-of-the-art large language model for coding, 2023.
[8] W. Ma, S. Liu, Z. Lin, W. Wang, Q. Hu, Y. Liu, C. Zhang, L. Nie, L. Li, Y. Liu, LMs: Understanding code syntax and semantics for code analysis, 2024, http://dx.doi.org/10.48550/arXiv.2305.12138, arXiv:2305.12138 [cs].
[9] B. Berabi, A. Gronskiy, V. Raychev, G. Sivanrupan, V. Chibotaru, M. Vechev, DeepCode AI fix: Fixing security vulnerabilities with large language models, 2024, http://dx.doi.org/10.48550/arXiv.2402.13291, arXiv:2402.13291 [cs].
[10] M. Fu, C. Tantithamthavorn, T. Le, Y. Kume, V. Nguyen, D. Phung, J. Grundy, AIBugHunter: A practical tool for predicting, classifying and repairing software vulnerabilities, Empir. Softw. Eng. 29 (1) (2023) 1–33, http://dx.doi.org/10.1007/s10664-023-10346-3, Number: 1 Publisher: Springer.
[11] H. Hanif, S. Maffeis, VulBERTa: Simplified source code pre-training for vulnerability detection, in: 2022 International Joint Conference on Neural Networks, IJCNN, in: Proceedings of the International Joint Conference on Neural Networks, vol. 2022-July, Institute of Electrical and Electronics Engineers Inc., Padua, Italy, 2022, pp. 1–8, http://dx.doi.org/10.1109/IJCNN55064.2022.9892280.
[12] Z. Zheng, K. Ning, Y. Wang, J. Zhang, D. Zheng, M. Ye, J. Chen, A survey of large language models for code: Evolution, benchmarking, and future trends, 2024, http://dx.doi.org/10.48550/arXiv.2311.10372, arXiv:2311.10372 [cs].
[13] M. Chen, J. Tworek, H. Jun, et al., Evaluating large language models trained on code, 2021, http://dx.doi.org/10.48550/arXiv.2107.03374, arXiv:2107.03374 [cs].
[14] M.M. Abdollahpour, M. Ashtiani, F. Bakhshi, Automatic software code repair using deep learning techniques, Softw. Qual. J. (2023) http://dx.doi.org/10.1007/s11219-023-09653-1.
[15] Q. Zhang, C. Fang, W. Sun, Y. Liu, T. He, X. Hao, Z. Chen, APPT: Boosting automated patch correctness prediction via fine-tuning pre-trained models, IEEE Trans. Softw. Eng. (2023) http://dx.doi.org/10.1109/TSE.2024.3354969.
[16] A. Silva, N. Saavedra, M. Monperrus, GitBug-Java: A reproducible benchmark of recent Java bugs (MSR'24), 2024, http://dx.doi.org/10.48550/arXiv.2402.02961, arXiv:2402.02961 [cs].

[17] V. Csuvik, L. Vidács, FixJS: A Dataset of Bug-fixing JavaScript Commits, IEEE Computer Society, 2022, pp. 712–716, http://dx.doi.org/10.1145/3524842.3528480.

[18] T.-T. Nguyen, X.-Y. Zhang, P. Arcaini, F. Ishikawa, H.D. Vo, Automated program repair for variability bugs in software product line systems, J. Syst. Softw. (2024) 112152, http://dx.doi.org/10.1016/j.jss.2024.112152.

[19] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, D. Phung, VulRepair: a T5-based automated software vulnerability repair, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, in: ESEC/FSE 2022, Association for Computing Machinery, New York, NY, USA, 2022, pp. 935–947, http://dx.doi.org/10.1145/3540250.3549098.

[20] Q. Zhang, C. Fang, B. Yu, W. Sun, T. Zhang, Z. Chen, Pre-trained model-based automated software vulnerability repair: How far are we? IEEE Trans. Dependable Secur. Comput. 21 (04) (2024) 1–18, http://dx.doi.org/10.1109/TDSC.2023.3308897, Publisher: IEEE Computer Society.

[21] Y. Wang, W. Wang, S. Joty, S.C.H. Hoi, CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 2021, http://dx.doi.org/10.48550/arXiv.2109.00859, arXiv:2109.00859 [cs].

[22] Y. Wang, H. Le, A.D. Gotmare, N.D.Q. Bui, J. Li, S.C.H. Hoi, CodeT5+: Open code large language models for code understanding and generation, 2023, http://dx.doi.org/10.48550/arXiv.2305.07922, URL http://arxiv.org/abs/2305.07922. arXiv:2305.07922 [cs].

[23] B. Rozière, J. Gehring, F. Gloeckle, et al., Code llama: Open foundation models for code, 2024, http://dx.doi.org/10.48550/arXiv.2308.12950, arXiv:2308.12950 [cs].

[24] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, 2020, http://dx.doi.org/10.48550/arXiv.2002.08155, arXiv:2002.08155 [cs].

[25] D. Guo, S. Ren, S. Lu, et al., GraphCodeBERT: Pre-training code representations with data flow, in: International Conference on Learning Representations, OpenReview.net, Virtual Event, Austria, 2021, pp. 1–18.

[26] Y. Li, D. Choi, J. Chung, et al., Competition-level code generation with AlphaCode, Science 378 (6624) (2022) 1092–1097, http://dx.doi.org/10.1126/science.abq1158, Publisher: American Association for the Advancement of Science.

[27] F.F. Xu, U. Alon, G. Neubig, V.J. Hellendoorn, A systematic evaluation of large language models of code, 2022, http://dx.doi.org/10.48550/arXiv.2202.13169, arXiv:2202.13169 [cs].

[28] W.U. Ahmad, S. Chakraborty, B. Ray, K.-W. Chang, Unified pre-training for program understanding and generation, 2021, http://dx.doi.org/10.48550/arXiv.2103.06333, URL http://arxiv.org/abs/2103.06333. arXiv:2103.06333 [cs].

[29] A. Kanade, P. Maniatis, G. Balakrishnan, K. Shi, Learning and evaluating contextual embedding of source code, 2020, http://dx.doi.org/10.48550/arXiv.2001.00059, arXiv:2001.00059 [cs].

[30] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, C. Xiong, CodeGen: An open large language model for code with multi-turn program synthesis, 2023, http://dx.doi.org/10.48550/arXiv.2203.13474, arXiv:2203.13474 [cs].

[31] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P.J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, J. Mach. Learn. Res. 21 (140) (2020) 1–67.

[32] Meta, Codellama (code llama), 2024, URL https://huggingface.co/codellama. (Accessed 26 August 2024).

[33] K. Huang, S. Yang, H. Sun, C. Sun, X. Li, Y. Zhang, Repairing security vulnerabilities using pre-trained programming language models, in: 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops, DSN-W, 2022, pp. 111–116, http://dx.doi.org/10.1109/DSN-W54100.2022.00027, ISSN: 2325-6664.

[34] N. Jiang, K. Liu, T. Lutellier, L. Tan, Impact of code language models on automated program repair, 2023, http://dx.doi.org/10.48550/arXiv.2302.05020, arXiv:2302.05020 [cs].

[35] Q. Zhang, C. Fang, T. Zhang, B. Yu, W. Sun, Z. Chen, Gamma: Revisiting template-based automated program repair via mask prediction, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, 2023, pp. 535–547, http://dx.doi.org/10.1109/ASE56229.2023.00063, ISSN: 2643-1572.

[36] W. Wang, Y. Wang, S. Joty, S.C. Hoi, RAP-gen: Retrieval-augmented patch generation with CodeT5 for automatic program repair, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, in: ESEC/FSE 2023, Association for Computing Machinery, New York, NY, USA, 2023, pp. 146–158, http://dx.doi.org/10.1145/3611643.3616256.

[37] C.L. Goues, M. Pradel, A. Roychoudhury, Automated program repair, Commun. ACM 62 (12) (2019) 56–65, http://dx.doi.org/10.1145/3318162.

[38] Z. Chen, S. Kommrusch, M. Monperrus, Neural transfer learning for repairing security vulnerabilities in c code, IEEE Trans. Softw. Eng. 49 (1) (2023) 147–165, http://dx.doi.org/10.1109/TSE.2022.3147265, Conference Name: IEEE Transactions on Software Engineering.

[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, 2023, http://dx.doi.org/10.48550/arXiv.1706.03762, arXiv:1706.03762 [cs].

[40] X. Zhou, K. Kim, B. Xu, D. Han, D. Lo, Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources, in: 2024 IEEE/ACM 46th International Conference on Software Engineering, ICSE, 2024, pp. 1071–1083, http://dx.doi.org/10.1145/3597503.3639222, ISSN: 1558-1225.

[41] D. de Fitero-Dominguez, E. Garcia-Lopez, A. Garcia-Cabot, J.-J. Martinez-Herraiz, Enhanced automated code vulnerability repair using large language models, Eng. Appl. Artif. Intell. 138 (2024) 109291, http://dx.doi.org/10.1016/j.engappai.2024.109291.

[42] A.Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D.S. Chaplot, D.d.l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L.R. Lavaud, M.-A. Lachaux, P. Stock, T.L. Scao, T. Lavril, T. Wang, T. Lacroix, W.E. Sayed, Mistral 7B, 2023, http://dx.doi.org/10.48550/arXiv.2310.06825, arXiv:2310.06825 [cs].

[43] G. Bhandari, A. Naseer, L. Moonen, CVEfixes: automated collection of vulnerabilities and their fixes from open-source software, in: Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering, in: PROMISE 2021, Association for Computing Machinery, New York, NY, USA, 2021, pp. 30–39, http://dx.doi.org/10.1145/3475960.3475985.

[44] J. Fan, Y. Li, S. Wang, T.N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE summaries, in: Proceedings of the 17th International Conference on Mining Software Repositories, MSR '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 508–512, http://dx.doi.org/10.1145/3379597.3387501.

[45] T.B. Brown, B. Mann, N. Ryder, et al., Language models are few-shot learners, 2020, http://dx.doi.org/10.48550/arXiv.2005.14165, arXiv:2005.14165 [cs].

[46] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, C. Sutton, Program synthesis with large language models, 2021, http://dx.doi.org/10.48550/arXiv.2108.07732, arXiv:2108.07732 [cs].

[47] C. Hughes, A brief overview of cross entropy loss, 2025, URL https://medium.com/@chris.p.hughes10/a-brief-overview-of-cross-entropy-loss-523aa56b75d5. (Accessed 15 February 025).

[48] Salesforce, Salesforce/codet5-base · hugging face, 2024, URL https://huggingface.co/Salesforce/codet5-base. (Accessed 15 February 2025).

[49] MetaAI, codellama/CodeLlama-7b-instruct-hf · hugging face, 2024, URL https://huggingface.co/codellama/CodeLlama-7b-Instruct-hf. (Accessed 14 February 2025).

[50] G. Bhandari, N. Gavric, An incremental lightweight method for vulnerability data collection for security patches, 2024, http://dx.doi.org/10.21203/rs.3.rs-4508582/v1.

[51] A. Silva, S. Fang, M. Monperrus, RepairLLaMA: Efficient representations and fine-tuned adapters for program repair, 2024, http://dx.doi.org/10.48550/arXiv.2312.15698, arXiv:2312.15698 [cs].

[52] T.T. Vu, T.-D. Bui, T.-D. Do, T.-T. Nguyen, H.D. Vo, S. Nguyen, Automated description generation for software patches, Inf. Softw. Technol. 177 (2025) 107543, http://dx.doi.org/10.1016/j.infsof.2024.107543.

[53] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, S. Ma, CodeBLEU: a method for automatic evaluation of code synthesis, 2020, http://dx.doi.org/10.48550/arXiv.2009.10297, arXiv:2009.10297 [cs].

[54] C.-Y. Lin, ROUGE: A package for automatic evaluation of summaries, in: Text Summarization Branches Out, Association for Computational Linguistics, Barcelona, Spain, 2004, pp. 74–81.

[55] NA, ROUGE - a hugging face space by evaluate-metric, 2025, URL https://huggingface.co/spaces/evaluate-metric/rouge. (Accessed 16 February 2025).

[56] Y. Zhang, Y. Xiao, M.M.A. Kabir, D.D. Yao, N. Meng, Example-based vulnerability detection and repair in java code, in: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension, ICPC '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 190–201, http://dx.doi.org/10.1145/3524610.3527895.

[57] D. Wang, M. Galster, M. Morales-Trujillo, A systematic mapping study of bug reproduction and localization, Inf. Softw. Technol. 165 (2024) 107338, http://dx.doi.org/10.1016/j.infsof.2023.107338.

[58] N. Dissanayake, A. Jayatilaka, M. Zahedi, M.A. Babar, An empirical study of automation in software security patch management, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, ASE '22, Association for Computing Machinery, New York, NY, USA, 2023, pp. 1–13, http://dx.doi.org/10.1145/3551349.3556969.

[59] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9) (2022) 3280–3296, http://dx.doi.org/10.1109/TSE.2021.3087402, Conference Name: IEEE Transactions on Software Engineering.

[60] H. Tian, K. Liu, A.K. Kaboré, A. Koyuncu, L. Li, J. Klein, T.F. Bissyandé, Evaluating representation learning of code changes for predicting patch correctness in program repair, in: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, ASE '20, Association for Computing Machinery, New York, NY, USA, 2021, pp. 981–992, http://dx.doi.org/10.1145/3324884.3416532.

[61] T. Durieux, F. Madeiral, M. Martinez, R. Abreu, Empirical review of java program repair tools: a large-scale experiment on 2,141 bugs and 23,551 repair attempts, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, in: ESEC/FSE 2019, Association for Computing Machinery, New York, NY, USA, 2019, pp. 302–313, http://dx.doi.org/10.1145/3338906.3338911.

[62] M. Hafiz, M. Fang, Game of detections: how are security vulnerabilities discovered in the wild? Empir. Softw. Eng. 21 (5) (2016) 1920–1959, http://dx.doi.org/10.1007/s10664-015-9403-7.

[63] Q.-N. Phung, M. Kim, E. Lee, Identifying incorrect patches in program repair based on meaning of source code, IEEE Access 10 (2022) 12012–12030, http://dx.doi.org/10.1109/ACCESS.2022.3145983, Conference Name: IEEE Access.

[64] N. Jiang, K. Liu, T. Lutellier, L. Tan, Impact of code language models on automated program repair, in: Proceedings of the 45th International Conference on Software Engineering, ICSE '23, IEEE Press, Melbourne, Victoria, Australia, 2023, pp. 1430–1442, http://dx.doi.org/10.1109/ICSE48619.2023.00125.

[65] Q. Herreros, T. Veasey, T. Papoikonomou, RAG evaluation metrics: A journey through metrics, 2023, URL https://www.elastic.co/search-labs/blog/evaluating-rag-metrics.

[66] S. Yeo, Y.-S. Ma, S.C. Kim, H. Jun, T. Kim, Framework for evaluating code generation ability of large language models, ETRI J. 46 (1) (2024) 106–117, http://dx.doi.org/10.4218/etrij.2023-0357, URL https://onlinelibrary.wiley.com/doi/abs/10.4218/etrij.2023-0357. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.4218/etrij.2023-0357.

[67] Qwen/Qwen2.5-coder-32b-instruct · hugging face, 2024, URL https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct.

[68] Deepseek-ai/deepseek-coder-33b-instruct · hugging face, 2024, URL https://huggingface.co/deepseek-ai/deepseek-coder-33b-instruct.

[69] codellama/CodeLlama-70b-Instruct-hf · Hugging Face. URL https://huggingface.co/codellama/CodeLlama-70b-Instruct-hf.